

EXPERIMENT - 25

1. Problem Statement

Write a C program that displays a menu and uses a switch statement to print the "Decreasing pattern" selected by the user. The patterns have to be made with nested loops without using arrays, and each pattern may be written in a separate parameterless function with no return value.

- **Decreasing right-angled triangle**, input the number of rows n and print a decreasing right-angled triangle using asterisks.
- **Decreasing natural-number pattern**, input n . The first row has the natural numbers from 1 to n , and every next row prints one value less.
- **Decreasing repeated-number pattern**, input n . The first row starts with n and the value drops by one in every next row. Each value is printed as many times as its own value.
- **Reverse-number triangle**, input n . Every row prints natural numbers in descending order, starting from the value equal to the number of elements in that row and ending at 1.

2. Algorithm

Main Program

Step 1: START

Step 2: Display the menu: 1. Decreasing right-angled triangle, 2. Decreasing natural-number pattern, 3. Decreasing repeated-number pattern, 4. Reverse-number triangle.

Step 3: Read choice.

Step 4: Using a switch statement on choice:

 If choice is 1, call `decreasingTriangle()`.

 If choice is 2, call `decreasingNaturalNumber()`.

 If choice is 3, call `decreasingRepeatedNumber()`.

 If choice is 4, call `reverseNumberTriangle()`.

Step 5: STOP

Function `decreasingTriangle()`

Step 1: Read n .

Step 2: Repeat for $i = 1$ to n :

 Repeat for $j = 1$ to $n - i + 1$: print an asterisk.

 Move to a new line.

Step 3: Return to the calling function.

Function `decreasingNaturalNumber()`

Step 1: Read n .

Step 2: Repeat for $i = 1$ to n :

 Repeat for $j = 1$ to $n - i + 1$: print the value of j .

 Move to a new line.

Step 3: Return to the calling function.

Function decreasingRepeatedNumber()

Step 1: Read n.

Step 2: Repeat for i = 1 to n:

Set value = n - i + 1.

Repeat for j = 1 to value: print value.

Move to a new line.

Step 3: Return to the calling function.

Function reverseNumberTriangle()

Step 1: Read n.

Step 2: Repeat for i = 1 to n:

Repeat for j = n - i + 1 down to 1: print the value of j.

Move to a new line.

Step 3: Return to the calling function.

3. Test Case Table

#	Menu Choice	Input	Expected Output	Actual Output	Result
1	1. Decreasing triangle	n = 4	Rows of 4, 3, 2 and 1 asterisks	Same	PASS
2	2. Decreasing natural number	n = 4	1 2 3 4 / 1 2 3 / 1 2 / 1	Same	PASS
3	3. Decreasing repeated number	n = 4	4 4 4 4 / 3 3 3 / 2 2 / 1	Same	PASS
4	4. Reverse-number triangle	n = 4	4 3 2 1 / 3 2 1 / 2 1 / 1	Same	PASS
5	1. Decreasing triangle	n = 1	A single asterisk, the smallest triangle	Same	PASS
6	4. Reverse-number triangle	n = 6	Six rows, the first one being 6 5 4 3 2 1	Same	PASS

4. Source Code

decreasing.c

```
/* Menu driven program that uses a switch statement to print the
   decreasing pattern chosen by the user. Each pattern is written in
   its own function that takes nothing and returns nothing, and every
   pattern is made with nested loops. No arrays are used. */
```

```
#include <stdio.h>
```

```
void decreasingTriangle(void);
void decreasingNaturalNumber(void);
void decreasingRepeatedNumber(void);
void reverseNumberTriangle(void);
```

```
int main(void)
{
    int choice;

    printf("1. Decreasing right-angled triangle\n");
    printf("2. Decreasing natural-number pattern\n");
    printf("3. Decreasing repeated-number pattern\n");
    printf("4. Reverse-number triangle\n");
    printf("Enter your choice: ");
    scanf("%d", &choice);

    switch (choice) {
        case 1: decreasingTriangle();      break;
        case 2: decreasingNaturalNumber(); break;
        case 3: decreasingRepeatedNumber(); break;
        case 4: reverseNumberTriangle();   break;
    }

    return 0;
}
```

```
/* Row 1 has n asterisks and every next row has one less. The count
   for row i works out to n - i + 1. */
```

```
void decreasingTriangle(void)
{
    int n, i, j;

    printf("Enter the number of rows: ");
    scanf("%d", &n);

    for (i = 1; i <= n; i++) {
        for (j = 1; j <= n - i + 1; j++)
            printf("* ");
        printf("\n");
    }
}
```

```
/* Same shrinking count, but the numbers 1, 2, 3 ... are printed
   instead of asterisks. */
```

```
void decreasingNaturalNumber(void)
{
    int n, i, j;

    printf("Enter the number of rows: ");
    scanf("%d", &n);

    for (i = 1; i <= n; i++) {
        for (j = 1; j <= n - i + 1; j++)
            printf("%d ", j);
        printf("\n");
    }
}
```

```
/* Row 1 prints n, row 2 prints n - 1, and so on. The value and the
   number of times it has to be printed are the same, so one variable
```

```

    does both jobs here. */
void decreasingRepeatedNumber(void)
{
    int n, i, j, value;

    printf("Enter the number of rows: ");
    scanf("%d", &n);

    for (i = 1; i <= n; i++) {
        value = n - i + 1;
        for (j = 1; j <= value; j++)
            printf("%d ", value);
        printf("\n");
    }
}

/* Each row counts down to 1, starting from the number of elements in
   that row, so the inner loop runs backwards. */
void reverseNumberTriangle(void)
{
    int n, i, j;

    printf("Enter the number of rows: ");
    scanf("%d", &n);

    for (i = 1; i <= n; i++) {
        for (j = n - i + 1; j >= 1; j--)
            printf("%d ", j);
        printf("\n");
    }
}

```

5. Compilation and Output

I compiled it with `gcc -Wall -Wextra decreasing.c -o decreasing` so that even the small warnings would show up, and it compiled clean in the first attempt itself. The screenshots below are of the runs I did on my own machine.

Patterns 1 and 2

```
adilshakil — zsh — 120x31
adilshakil@ADILs-MacBook-Air MyAssignment % gcc -Wall -Wextra decreasing.c -o decreasing
adilshakil@ADILs-MacBook-Air MyAssignment % ./decreasing
1. Decreasing right-angled triangle
2. Decreasing natural-number pattern
3. Decreasing repeated-number pattern
4. Reverse-number triangle
Enter your choice: 1
Enter the number of rows: 4
* * * *
* * *
* *
*

adilshakil@ADILs-MacBook-Air MyAssignment % ./decreasing
1. Decreasing right-angled triangle
2. Decreasing natural-number pattern
3. Decreasing repeated-number pattern
4. Reverse-number triangle
Enter your choice: 2
Enter the number of rows: 4
1 2 3 4
1 2 3
1 2
1
█
```

Patterns 3 and 4

```
adilshakil — zsh — 120x31
adilshakil@ADILs-MacBook-Air MyAssignment % ./decreasing
1. Decreasing right-angled triangle
2. Decreasing natural-number pattern
3. Decreasing repeated-number pattern
4. Reverse-number triangle
Enter your choice: 3
Enter the number of rows: 4
4 4 4 4
3 3 3
2 2
1

adilshakil@ADILs-MacBook-Air MyAssignment % ./decreasing
1. Decreasing right-angled triangle
2. Decreasing natural-number pattern
3. Decreasing repeated-number pattern
4. Reverse-number triangle
Enter your choice: 4
Enter the number of rows: 4
4 3 2 1
3 2 1
2 1
1
█
```

Smallest Value of n, and a Bigger Triangle

```
adilshakil@ADILs-MacBook-Air MyAssignment % ./decreasing
1. Decreasing right-angled triangle
2. Decreasing natural-number pattern
3. Decreasing repeated-number pattern
4. Reverse-number triangle
Enter your choice: 1
Enter the number of rows: 1
*

adilshakil@ADILs-MacBook-Air MyAssignment % ./decreasing
1. Decreasing right-angled triangle
2. Decreasing natural-number pattern
3. Decreasing repeated-number pattern
4. Reverse-number triangle
Enter your choice: 4
Enter the number of rows: 6
6 5 4 3 2 1
5 4 3 2 1
4 3 2 1
3 2 1
2 1
1
█
```

All six test cases came out right and the four patterns match the sample outputs given in the question exactly. The part I had to think about was the row length, because here it shrinks instead of growing. Once I worked out that row i has $n - i + 1$ elements, all four functions became the same loop with only the printed value changing.